Ah, cette ligne-là… elle a l'air simple, mais elle fait **beaucoup de travail intelligent**.
Je vais te l'expliquer **clairement**, **pas à pas**, et surtout **dans quel contexte elle est utilisée**.

🔍 La ligne à expliquer

```
$done  = isset($data['done']) ? (int)$data['done'] : (int)$task['done'];
```

🧠 Explication simple et claire

Cette ligne signifie :

Si le JSON envoyé par le frontend contient `done`, alors on utilise cette valeur.
Sinon, on garde la valeur `done` déjà présente dans la base.

C'est une **opération ternaire**, c'est-à-dire une condition en une seule ligne.

🧩 Décomposition ligne par ligne

1️⃣ `isset($data['done'])`

On vérifie si le frontend a envoyé une valeur `done`.

- Si oui → on utilise la nouvelle valeur
- Si non → on garde l'ancienne valeur

2️⃣ `(int)$data['done']`

Si `done` existe dans `$data` :

- on le convertit en **entier**
- pour éviter `"0"` ou `"1"` (string)
- et garantir `0` ou `1` (number)

➡️ C'est important pour Vue, sinon toutes les tâches apparaissent barrées.

---

3 `(int)$task['done']`

Si le frontend **n'a pas envoyé** de valeur `done` :

- on garde la valeur actuelle de la tâche dans la base
- et on la convertit aussi en entier

---

4 Le ternaire complet

```
$done = condition ? valeur_si_vrai : valeur_si_faux;
```

Donc :

- **Si** `$data['done']` existe → `$done = (int)$data['done']`
- **Sinon** → `$done = (int)$task['done']`

🎯 Pourquoi cette ligne est importante ?

Parce qu'elle permet :

✅ 1. De mettre à jour seulement ce que le frontend envoie

Si tu fais un PUT mais que tu n'envoies pas `done`,
alors la tâche **ne perd pas son état**.

✅ 2. D'éviter les bugs Vue (string vs number)

Vue considère `"0"` comme **true**, donc tâche barrée.
Avec `(int)`, tu garantis :

- `0` → non barré
- `1` → barré

✅ 3. De rendre ton API plus robuste

Même si le frontend oublie un champ,
ton API ne casse pas.

🧭 Exemple concret

📌 Cas 1 : le frontend envoie

```
{ "title": "Faire les courses", "done": 1 }
```

Alors :

```
isset($data['done']) = true
$done = (int)1 = 1
```

➡️ La tâche devient terminée.

📌 Cas 2 : le frontend envoie

```
{ "title": "Faire les courses" }
```

Alors :

```
isset($data['done']) = false
$done = (int)$task['done']  // on garde l'ancien état
```

➡️ La tâche garde son état précédent.

🎉 Résumé ultra simple

Cette ligne :

- vérifie si `done` est envoyé
- convertit en entier
- garde l'ancienne valeur si rien n'est envoyé
- évite les bugs d'affichage dans Vue
- rend ton API plus solide

Si tu veux, je peux aussi t'expliquer :

- la ligne équivalente **sans ternaire**
- comment cette valeur est utilisée dans ton `UPDATE` SQL
- comment Vue interprète `done` côté frontend

Tu veux que je t'explique la version sans ternaire ?